

# Playbook Kiro Build Day

Qué es Daily Brief hoy, qué le sacamos para el prototipo “**Preguntale a El País**” y qué hace cada uno, hora por hora. Pensado para imprimir y tener en la mesa.

---

## LA IDEA EN DOS ORACIONES

### Un lector le pregunta algo a El País y recibe una respuesta corta, con las notas que la respaldan

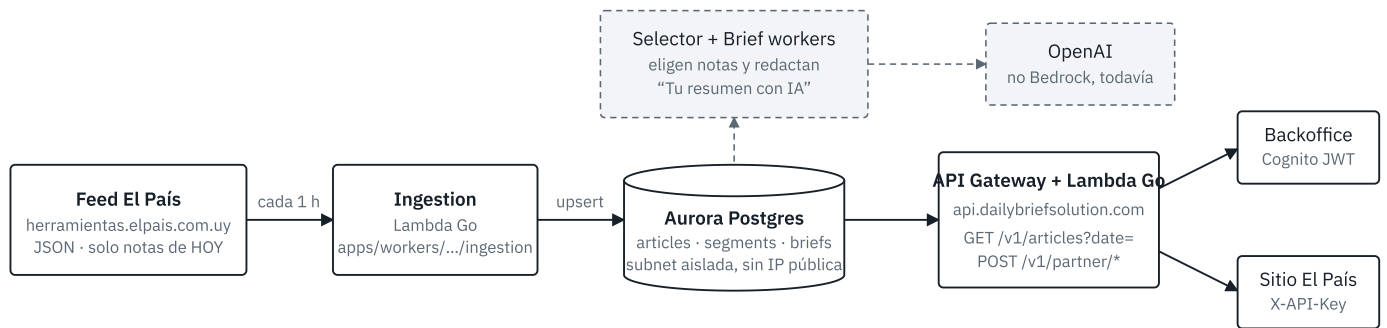
Igual que *Ask The Post AI* del Washington Post: el modelo responde **únicamente** con lo publicado por El País, cita cada nota con su link y, si no hay cobertura, lo dice. Es una nueva forma de consumir el diario (más cerca de una app que de una portada) y un embudo natural hacia la suscripción: la respuesta es gratis, la nota completa es para suscriptores.

La ventaja competitiva que tenemos sobre cualquier otro equipo es que **el contenido ya está**: Daily Brief ingesta todas las notas del día desde hace meses. No hay que inventar datos de prueba.

## MAPA · DAILY BRIEF HOY

### Qué hay funcionando y de dónde sale cada cosa

Daily Brief es un monorepo (pnpm + Turborepo) que arma resúmenes por segmento de lectores para El País. Todo corre en la cuenta AWS `178042202224`, región `us-east-1`, con dos entornos: **prod** vivo y **dev** apagado desde junio por costos.



- ☐ Lo que reutiliza el prototipo: el feed (notas de hoy) y la API de artículos (histórico con cuerpo completo)
- ☒ Pipeline de briefs: sigue corriendo, no lo tocamos

El contenido entra una vez por hora desde el feed y queda en Aurora. Como la base no es accesible desde afuera, el prototipo no la toca: lee las notas por la API con un usuario admin, o directo del feed.

## Dónde está cada cosa en el repo

| QUÉ              | DÓNDE                                                                     | PARA QUÉ NOS SIRVE                                                                                                                                |
|------------------|---------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Ingesta del feed | apps/workers/internal/ingestion/handler.go                                | Muestra el formato del feed (titulo, bajada, cuerpo_texto, link, keywords, categorySlug). Es el contrato de datos.                                |
| Modelo de nota   | apps/api/internal/models/article.go · packages/db/migrations/001_init.sql | Campos que vamos a indexar: title, deck, bodyText, link, category, keywords, feedDate.                                                            |
| API de artículos | apps/api/internal/handlers/articles.go                                    | GET /v1/articles?date=YYYY-MM-DD lista hasta 500 notas del día (sin cuerpo). GET /v1/articles/{id} trae el cuerpo. Requiere Cognito, grupo admin. |
| API partner      | docs/PARTNER_INTEGRATION_UPDATED.md                                       | Briefs y carruseles del día con X-API-Key . Útil para el pitch (“ya integramos al sitio así”), no para el corpus: no devuelve cuerpo.             |

| QUÉ                 | DÓNDE                                                                                               | PARA QUÉ NOS SIRVE                                                                                                          |
|---------------------|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Prompts editoriales | <code>apps/workers/internal/prompts/prompts.go</code> · <code>input/Prompt - Borrador (1).md</code> | Reglas “cero invención, cada afirmación respaldada, link inline”. Se reciclan casi textuales para el prompt del agente.     |
| Cliente LLM actual  | <code>apps/workers/internal/ai/client.go</code>                                                     | Hoy habla con OpenAI (API compatible chat/completions). Argumento para el jurado: el prototipo es el primer paso a Bedrock. |
| Infra               | <code>infrastructure/lib/*.ts</code> · <code>scripts/deploy.sh</code>                               | CDK TypeScript con guardrails de cuenta/región. Referencia de estilo para lo que genere Kiro.                               |
| Diagrama existente  | <code>apps/backoffice/src/docs/arquitectura.md</code>                                               | Mermaid de toda la plataforma (v1.3). Pegar en mermaid.live para imprimirlo también.                                        |
| Segmentos           | <code>scripts/segments-prod-minimal.json</code>                                                     | Ejemplo del segmento <code>general</code> con topics. Sirve para “respuestas según tus intereses” si sobra tiempo.          |

#### Tres cosas que hay que saber antes de arrancar.

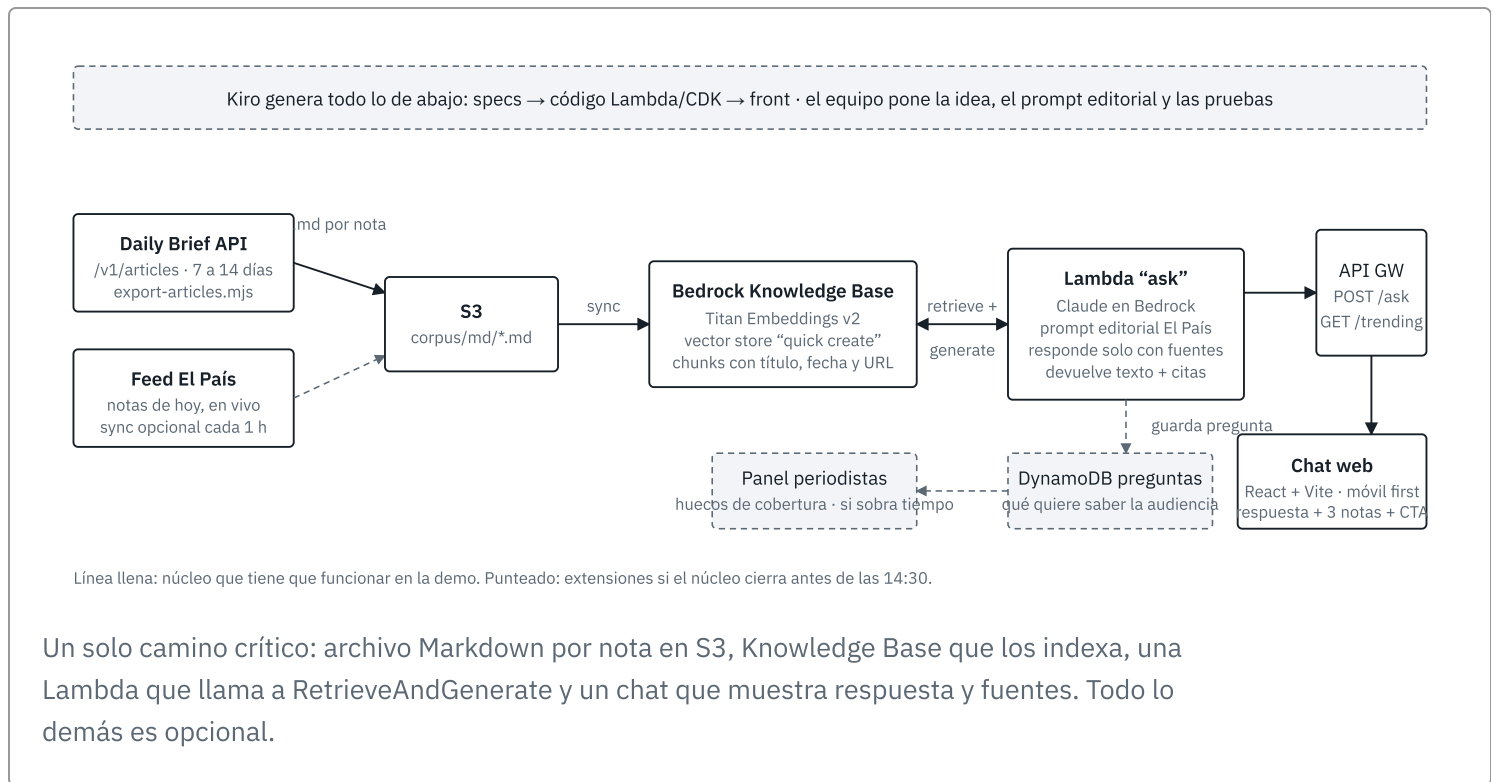
- **Dev está apagado.** `api-dev.dailybriefsolution.com` ni resuelve DNS. Todo lo que probemos es contra prod: solo lecturas.
- **El feed devuelve solo el día en curso.** A la madrugada viene vacío. El histórico con cuerpo sale por `GET /v1/articles`.
- **Daily Brief hoy usa OpenAI, no Bedrock.** El challenge exige Bedrock, así que el prototipo trae su propio modelo. No hay que tocar nada del pipeline actual.

#### ARQUITECTURA DEL PROTOTIPO

## Corpus afuera, Bedrock en el medio, un chat adelante

La decisión clave: **desacoplar el prototipo de la cuenta de Daily Brief**. Exportamos las notas a archivos esta noche y las subimos a S3 en la cuenta que nos den en el evento (o en

la nuestra si Bedrock ya está habilitado). Así no dependemos de VPCs, secretos ni permisos ajenos a las 10:30 de la mañana.



## Por qué Knowledge Base y no un RAG casero

- Se crea desde la consola en 10 a 15 minutos, indexa Markdown de S3 sin código y devuelve **citas con el archivo fuente**. Eso es exactamente lo que hay que mostrar en vivo.
- Puntúa alto en “uso creativo de Kiro + AWS”: Bedrock KB + Claude + Kiro generando la Lambda, la infra y el front.
- Plan B** si la KB falla o tarda: Lambda que carga `corpus.jsonl` en memoria, embebe con Titan Embeddings al arrancar y hace similitud coseno. Son unos cientos de notas, entra en memoria sin problema. Kiro lo escribe en 20 minutos.

## Vector store: elegir en el momento

Al crear la Knowledge Base, en “quick create” aparecen opciones según la cuenta. Preferir **S3 Vectors** si está disponible (barato, sin cluster). Si solo ofrece OpenSearch Serverless, usarlo igual y **borrarlo al terminar el día**: cobra por hora aunque esté vacío.

ANTES DE LAS 9:30

# Checklist de esta noche y de la mañana

- ☐ Instalar Kiro desde [kiro.dev](https://kiro.dev) y loguearse una vez (cuenta AWS Builder ID o la que indique el evento).

- ☐ Instalar AWS CLI: `brew install awscli`. Sin esto no se sube nada a S3 desde la terminal.
- ☐ Confirmar que alguien del equipo puede entrar al backoffice prod `app.dailybriefsolution.com` con un usuario del grupo admin. Es el mismo usuario que usa el script de exportación.
- ☐ Correr `node kiro-build-day/export-articles.mjs --days 14` y verificar que `corpus/manifest.json` tenga varios cientos de notas. Copiar la carpeta a un pendrive y a otra laptop.
- ☐ Averiguar si el evento da una cuenta AWS con Bedrock habilitado o si usamos la nuestra. Si es la nuestra, pedir a quien administre `178042202224` que verifique acceso a Claude y Titan Embeddings en Bedrock (Model access) hoy.
- ☐ Nombre y logo del equipo en [nova.amazon.com](https://nova.amazon.com). Imprimir el logo, definir remera o color común.
- ☐ Imprimir este playbook y el diagrama del prototipo (2 copias).
- ☐ Preparar 8 preguntas de demo sobre notas reales del corpus, incluida una sin cobertura para mostrar que el agente dice “El País no publicó sobre esto”.
- ☐ Alargue, cargadores, hotspot del celular por si el wifi falla.

## ROLES

# Cinco frentes para repartir

### Backend y datos

Sube el corpus a S3, crea la Knowledge Base, guía a Kiro para generar la Lambda “ask” y el API Gateway. Dueño del plan B.

### Producto y pitch

Define el prompt editorial, las reglas de negocio (solo El País, citas, CTA suscripción), arma el guion de 4 minutos y la historia de impacto.

### Front y equipo

Con Kiro genera el chat web (móvil first, estilo El País), logo, identidad, energía. Prueba el flujo como lectora.

### QA y preguntas

Curador del corpus: prueba las preguntas de demo, detecta alucinaciones, ajusta el prompt con producto. Cronometra.

### Enlace con el Solutions Architect + panel

Pegado al SA de AWS para desbloquear permisos de Bedrock y S3. Si el núcleo cierra, arma la tabla de preguntas y el panel de periodistas.

# Hora por hora

---

09:30

## Bienvenida y desayuno

- Conseguir las credenciales AWS del evento. Preguntar al SA: región con Bedrock, modelos habilitados, si se puede crear Knowledge Base.
- Abrir Kiro en las tres laptops y crear el proyecto vacío `pregunta-elpais`.

10:00

## Teoría e instrucciones

- Anotar qué modelo recomiendan y cómo prefieren que se despliegue (CDK, SAM, consola). Adaptar la spec a eso.

10:30

## Bloque 1 · el núcleo tiene que funcionar antes del almuerzo

- **10:30** Backend: `aws s3 mb + aws s3 sync corpus/md s3://.../notas/`. Crear la Knowledge Base desde la consola apuntando a ese bucket; lanzar el sync.
- **10:30** Producto y front: pegar en Kiro el prompt de arranque (abajo) y dejar que genere `requirements.md`, `design.md`, `tasks.md`. Revisar, no aceptar a ciegas.
- **11:00** Probar la KB desde la consola con 3 preguntas reales. Si responde con citas, la parte difícil está resuelta.
- **11:15** Kiro genera la Lambda “ask” (RetrieveAndGenerate, prompt editorial, formato de respuesta JSON con `answer`, `sources[]`) y el API Gateway. Desplegar.
- **12:00** Kiro genera el chat web contra ese endpoint. Primera demo de punta a punta, aunque sea fea.
- **12:45** Congelar alcance. Anotar lo que falta en dos listas: “necesario” y “si sobra tiempo”.

13:00

## Almuerzo

- Producto escribe el guion del pitch. El resto come sin abrir la laptop; la segunda mitad rinde más descansados.

14:00

## Bloque 2 · pulir y ensayar

- **14:00** Afinar prompt: rechazo elegante cuando no hay cobertura, respuesta en rioplatense, máximo 3 párrafos, siempre links.
- **14:30** Solo si el núcleo está estable: guardar preguntas en DynamoDB y endpoint `GET /trending` con las más repetidas. Es la semilla del panel de periodistas.
- **15:00** Ensayo completo con cronómetro, dos veces. Grabar un video de la demo con el celular por si el wifi muere en el escenario.
- **15:20** Limpiar pantalla, agrandar fuente del navegador, preparar las 8 preguntas en un archivo para copiar y pegar.

- Producto abre con el problema (60 s), front hace la demo en vivo (2 min), backend explica arquitectura y por qué escala (60 s), cierre con roadmap y pedido concreto (30 s).

#### PROMPT DE ARRANQUE PARA KIRO

## Lo que hay que pegarle a Kiro a las 10:30

Kiro trabaja mejor con una spec clara. Este texto le da contexto, restricciones y el orden de trabajo. Ajustar el modelo y la región a lo que diga el SA.

Proyecto: "Preguntale a El País". Prototipo para el Kiro Build Day (4 horas).

Contexto: El País (Uruguay) ya tiene un sistema llamado Daily Brief que ingesta todas las notas del diario. Exportamos las notas de los últimos 14 días como archivos Markdown (uno por nota; cada archivo empieza con título, fecha, sección y URL) y están en S3 en `s3://BUCKET/notas/`. Hay una Bedrock Knowledge Base creada sobre ese bucket con id `KB_ID` en la región `REGION`.

Objetivo: un chat web donde un lector pregunta en español y recibe una respuesta breve basada ÚNICAMENTE en notas de El País, con las notas citadas (título + URL). Si el corpus no cubre la pregunta, responder: "El País no publicó sobre esto en los últimos días" y sugerir 2 notas relacionadas si las hay.

Reglas del prompt del modelo (vienen del prompt editorial de Daily Brief):

- Cero invención: nada que no esté explícito en las notas recuperadas.
- Español rioplatense, tono editorial sobrio, máximo 3 párrafos.
- Cada afirmación respaldada por una fuente. No decir "la nota dice".
- Terminar con una invitación a leer la nota completa en El País.

Stack: Lambda en Node.js 20 (o Python 3.12) que llama a `bedrock-agent-runtime RetrieveAndGenerate` con el modelo `MODELO_CLAUDE` y la Knowledge Base `KB_ID`; API Gateway HTTP con `POST /ask {question}` que devuelve `{answer, sources:[{title,url,date,snippet}]}`; front React + Vite estático, móvil first, un solo campo de texto y chips con preguntas sugeridas; infra con AWS CDK en TypeScript. Guardar cada pregunta en una tabla DynamoDB (pregunta, timestamp, cantidad de fuentes) para un futuro panel de periodistas.

Orden de trabajo: 1) spec (requirements, design, tasks), 2) Lambda + CDK, deploy y prueba con curl, 3) front contra el endpoint real, 4) DynamoDB y GET `/trending` solo si todo lo anterior funciona. Priorizá que corra de punta a punta antes de agregar nada.

Cuando Kiro proponga la spec, leerla en voz alta entre dos y sacar todo lo que no aporte a la demo. Cada feature extra que quede en `tasks.md` es media hora menos de ensayo.

# La historia que se cuenta en cuatro minutos

1. **El problema.** Las nuevas generaciones no entran por la portada: preguntan. Hoy esa pregunta la responde un buscador o un chatbot genérico con información sin verificar, y El País no participa en esa conversación.
2. **La solución.** Un agente que responde solo con periodismo de El País, cita cada nota y lleva al lector a suscribirse para leer completo. Mostrar tres preguntas: una de actualidad, una de deportes o cultura, y una sin cobertura para demostrar que no inventa.
3. **Por qué funciona ya.** El contenido validado ya está: Daily Brief ingesta todo el diario cada hora. No es una maqueta con datos ficticios, son las notas de esta semana.
4. **Qué construimos con Kiro y AWS.** Knowledge Base sobre S3, Claude en Bedrock, Lambda y front generados con Kiro a partir de una spec. Cuatro horas, cero código escrito a mano.
5. **Impacto y siguiente paso.** Nuevo punto de contacto con lectores jóvenes, señal de qué quiere saber la audiencia para la redacción (el panel de preguntas), y un embudo directo a suscripción. Pedido: sesión con los SA para llevarlo a producción sobre la infra actual de Daily Brief.

## Cómo mapea al puntaje

- 25 % Prototipo funcional: la demo en vivo con preguntas reales y citas. Grabar un video de respaldo.
- 25 % Impacto de negocio: suscripciones, audiencia joven y datos para la redacción. Se cuenta con un ejemplo concreto.
- 20 % Uso creativo de Kiro + AWS: spec-driven con Kiro, Knowledge Base, Claude, DynamoDB para el loop con periodistas.
- 15 % Storytelling: problema, demo, arquitectura, roadmap. Ensayado dos veces.
- 15 % Espíritu de equipo: nombre, logo de Nova Canvas, color común, cinco personas con un rol claro cada una.

---

Preparado la noche del 3 al 4 de septiembre de 2026 a partir del repo `daily_brief`. El script de exportación está en `kiro-build-day/export-articles.mjs`.